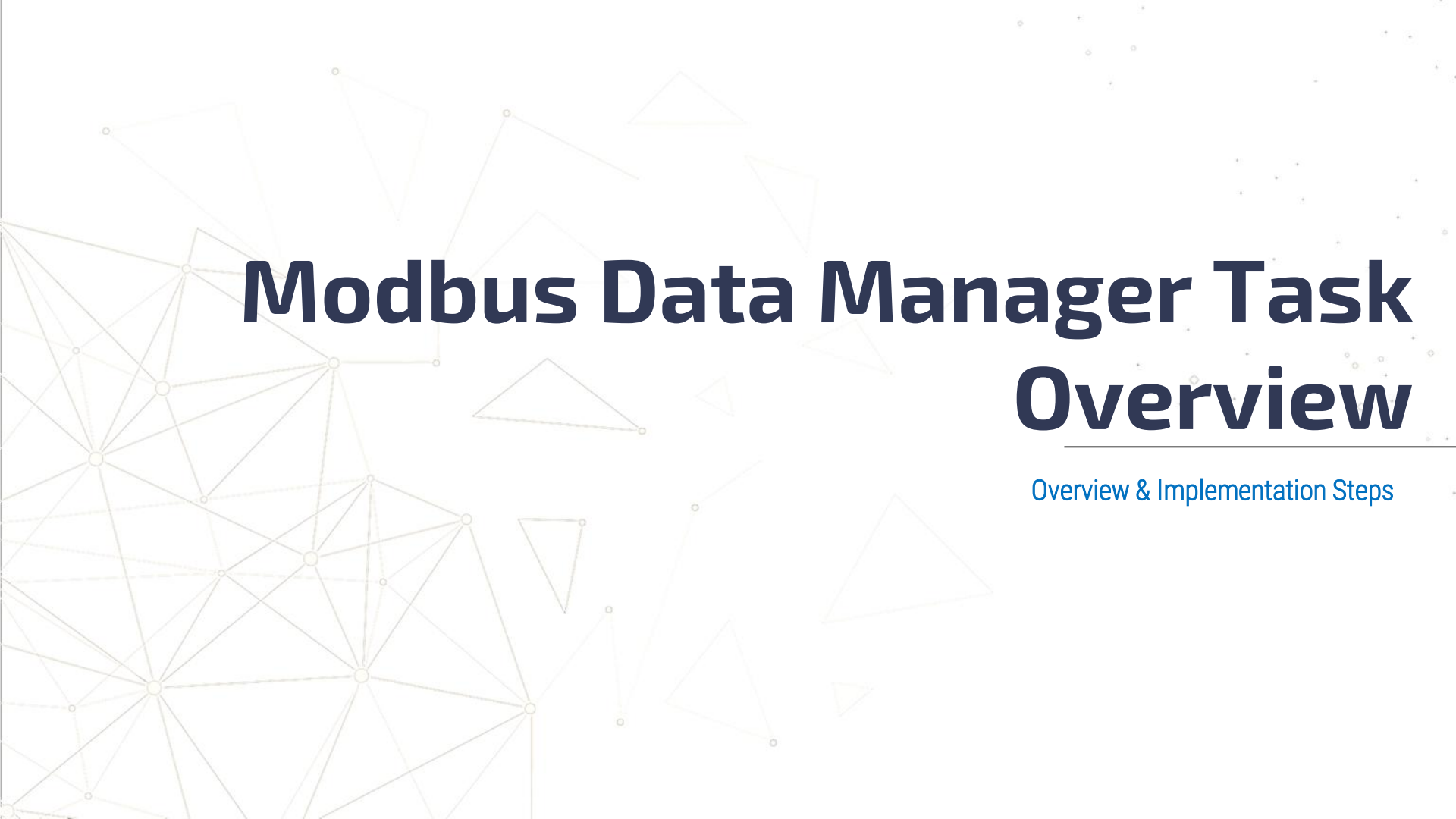




Modbus Data Manager Task Overview

Overview & Implementation Steps

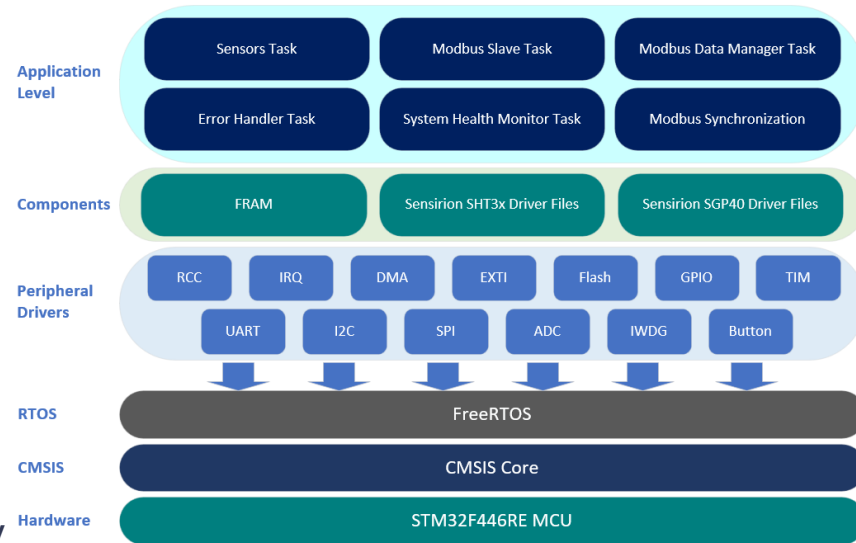
Modbus Data Manager Task Overview

- Modbus Data Manager Task Overview
- Section Implementation Steps

Modbus Data Manager Task Overview Part I

General

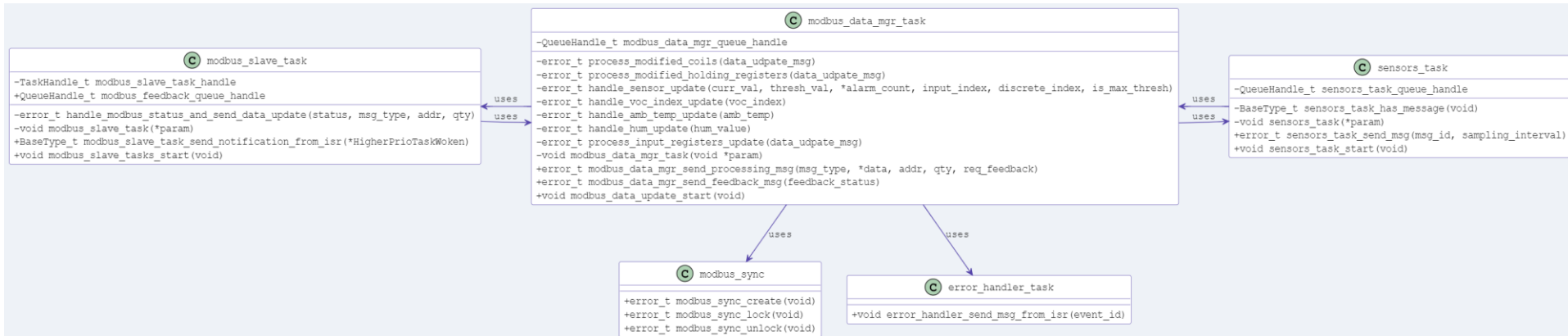
- **Purpose & Functionality:** Handles register updates from Sensors and Modbus Slave Tasks, ensuring synchronization of system state. Manages holding and input registers, with alarm counts and configuration settings stored in FRAM.
- **Project Significance:**
 - **Central Data Processing:** Applies register updates from sensors and Modbus master.
 - **Task Coordination:** Maintains data integrity by managing updates through queued messages, providing feedback when needed.
 - **FRAM Synchronization:** Writes alarm counts and updated holding registers to FRAM for persistence.



Modbus Data Manager Task Overview Part II

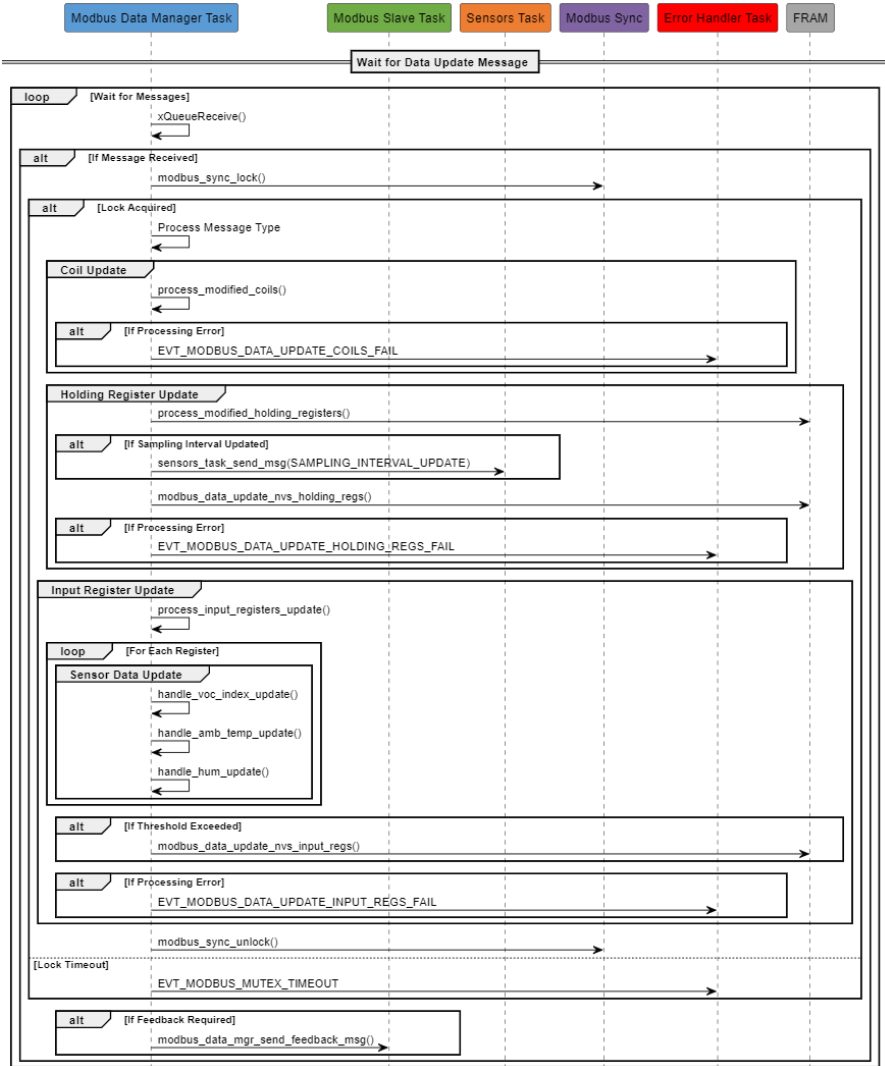
FreeRTOS Task Activities

- **Handles register updates** by processing messages from the **Modbus Slave Task** for changes to coils and holding registers, sending feedback when processing completes.
- **Processes sensor data** from the **Sensors Task**, updating discrete inputs based on VOC, ambient temperature, and humidity thresholds.
- **Ensures thread safety** using **mutex synchronization** to protect shared resources and prevent data corruption.
- **Error handling integration:** Sends messages to the **Error Handler Task** if issues occur during register updates or sensor processing.



Modbus Slave Task Overview Part III

- **Queue Processing:** Waits for messages from the Modbus Slave Task or Sensors Task to update registers.
- **Coil Updates:** Modifies coils and reports processing errors if any occur.
- **Holding Register Updates:** Processes changes, updates FRAM, and adjusts the sampling interval if modified.
- **Input Register Updates:** Handles sensor data updates, modifies discrete inputs, and checks alarm thresholds.
- **FRAM Storage:** Saves updated register values when necessary.
- **Error Handling:** Sends error events if processing fails.



Modbus Data Manager Task Files Preparation

Overview

- **Objective:** Prepare .c/.h files for managing Modbus data updates, ensuring synchronization and structured communication between tasks.
- **Key Steps:**
 - Create `modbus_data_mgr_task.h` with message types, queue structures, and function prototypes for processing data updates.
 - Implement `modbus_data_mgr_task.c` to handle queued Modbus updates, apply register modifications, manage synchronization, and send feedback to the Modbus Slave Task.
- **Project Significance:** Establishes a structured task for managing register updates, ensuring consistency and thread safety through a mutex.

DMA Driver Overview & Integration

Overview

- **Objective:** Integrate and review the custom DMA driver for SPI1, optimizing SPI RX/TX operations with interrupt-driven transfers and synchronization.
- **Key Steps:**
 - **Understand:** The role of DMA in handling SPI1 RX (Stream 2) and TX (Stream 3) transfers.
 - **Integrate and Review:** DMA initialization, stream configuration, and interrupt setup.
 - **Integrate and Review:** Stream disable handling with timeouts to prevent synchronization issues.
- **Project Significance:**
 - Ensures efficient SPI communication with FRAM using DMA, reducing CPU load.

SPI Driver Overview & Integration

Overview

- **Objective:** Integrate and review the custom SPI driver, focusing on initialization, configuration, and data transfer using interrupts and DMA with semaphore synchronization.
- **Key Steps:**
 - **Understand:** SPI driver structure, configuration, and supporting functions.
 - **Integrate and Review:** Header file definitions, structures, static inline functions and public function prototypes.
 - **Integrate and Review:** Source file functions and interrupt handlers.
- **Project Significance:**
 - Ensures proper SPI peripheral configuration, data transmission/reception, and synchronization in an RTOS environment.

SPI FRAM GPIO Overview & Initialization

Overview

- **Objective:** Review the GPIO configuration for SPI communication with FRAM, ensuring correct pin assignments, create the PORT / PIN definitions and initialize.
- **Key Steps:**
 - **Verify:** Configuration of FRAM Chip Select (CS), SPI SCK, MISO, and MOSI pins.
 - **Define:** SPI FRAM GPIO pin mappings in `gpio_defs.h`.
 - **Implement:** GPIO initialization in `gpio.c`, setting modes, alternate functions, and pull-up/down settings.
- **Project Significance:**
 - Ensures proper SPI communication with FRAM by correctly initializing GPIOs.

FRAM Review & Driver Development

Overview

- **Objective:** Review FRAM functionality, then implement initialization, read, and write operations using SPI and DMA.
- **Key Steps:**
 - **Setup & Integration:** Create `fram.c`, include `fram.h` and `spi.h`, and initialize SPI for FRAM communication.
 - **Write Operations:** Implement `fram_write_enable()`, `fram_write_disable()`, and `fram_write()` for data storage.
 - **Read Operations:** Implement `fram_read()`.
- **Project Significance:** Prepares the FRAM driver for seamless integration with Modbus and sensor data logging.

IRQ Priorities Update & FRAM Testing

Overview

- **Objective:** Set **SPI1 IRQ** and **DMA** priorities to ensure reliable FRAM transactions, followed by a basic FRAM read/write test for validation.
- **Key Steps:**
 - **Update IRQ Priorities:** Define SPI1 IRQ and DMA2 Stream 2 & 3 priorities in `irq.h`.
 - **Configure NVIC:** Apply settings in `irq.c` for correct interrupt handling.
 - **Verify FRAM Communication:** Run test in `main.c` to check SPI1 read/write.
- **Project Significance:**
 - Ensures stable FRAM transactions with SPI1 & DMA and validates FRAM functionality before full system integration.

System Event Group Implementation

Overview

- **Objective:** Introduce and implement a **FreeRTOS Event Group** for synchronizing the Modbus Slave and Sensors tasks, ensuring Modbus registers are initialized before sensor readings begin.
- **Key Steps:**
 - **Define Event Group:** Create `system_event_group` in `system_events.h`.
 - **Set Initialization Bit:** Modify `modbus_slave_task.c` to set `MODBUS_INITIALIZED_BIT` once Modbus registers are ready.
 - **Wait for Modbus:** Update `sensors_task.c` to wait for `MODBUS_INITIALIZED_BIT` before reading sensor data.
- **Project Significance:**
 - Ensures the Sensors Task starts only after Modbus initialization, preventing incorrect sampling intervals and maintaining system reliability.

Modbus Slave Task Expansion

Overview

- **Objective:** Expand the Modbus Slave Task to initialize registers, process Modbus function codes, and send updates to the Modbus Data Manager.
- **Key Steps:**
 - **Initialize Registers:** Load Holding and Input Registers from FRAM at startup.
 - **Data Manager Updates & Feedback:** Implement a helper function to transmit updates to the Data Manager and retrieve processing feedback for master responses.
- **Project Significance:**
 - Ensures the Modbus Slave Task can manage register updates and communicate with the Modbus Data Manager.

Sensors Task Expansion

Overview

- **Objective:** Modify the Sensors Task to retrieve the sampling interval from a Holding Register and send sensor data updates to the Modbus Data Manager.
- **Key Steps:**
 - **Retrieve Sampling Interval:** Read the interval from Holding Registers and convert it to RTOS tick units.
 - **Send Data Updates:** Transmit VOC, temperature, and humidity data to the Data Manager for Input Register updates.
- **Project Significance:**
 - Ensures sensor data is collected and updated at the correct intervals based on the Sensor Sampling Interval Modbus Holding Register.

Modbus Data Manager Task Final Development

Overview

- **Objective:** Finalize the Modbus Data Manager Task by implementing coils, holding registers, and input register processing functions to handle data updates, threshold checks, and alarm management.
- **Key Steps:**
 - **Coils & Holding Registers Processing:** Implement functions to update FRAM and trigger actions based on register modifications.
 - **Input Registers Processing:** Handle sensor data updates, perform threshold checks, and manage alarm states.
- **Project Significance:**
 - Ensures the Modbus Data Manager can process register updates, manage alarms, and maintain system consistency.

Modbus Data Manager & Application Testing

- **Objective:** Validate Modbus communication and register updates using Simply Modbus Master, ensuring proper interaction between the **Modbus Slave Task**, **Data Manager**, and **Sensors Task**.
- **Key Steps:**
 - **Verify Initialization & Input Register Updates:** Verify registers are initialized on startup and check updates triggered by the Sensors Task through the Data Manager.
 - **Modify Sampling Interval:** Use Function Code 6 to update the Sensors Sampling Interval via the Modbus Slave Task and Data Manager.
 - **Test Full Register Writes:** Use Function Code 16 to update multiple registers and verify persistence.
 - **Validate Discrete Inputs & Alarms:** Test alarm triggering by introducing VOC threshold changes.
 - **Confirm Persistence on Restart:** Restart the system to verify stored register values and alarm counts.
- **Project Significance:**
 - Ensures register updates, alarms, and data persistence function correctly, validating system behavior before deployment or further expansion.

The background features a complex network of thin, light-colored lines and small circles, forming various triangular shapes. Some of these shapes are filled with a very light yellow or green color. The overall effect is a subtle, technical, or network-like pattern.

Next: Modbus Data Manager Task Files Preparation
